Reporte de Pentest — Cliente A — firma legal

> **Nota sobre anonimización.** Este informe documenta un trabajo real, realizado con autorización del propietario. El nombre del cliente y los identificadores de su infraestructura se reemplazaron por marcadores (`Cliente A` , `cliente-a`) antes de publicarlo. Belona Cybersecurity no publica datos identificables de un cliente sin su autorización por escrito.

- **Fecha:** 2 de julio de 2026
- **Tipo:** Caja gris (grey-box), autorizado por el propietario
- **Alcance:** `https://cliente-a.web.app` + backend Firebase (Firestore, Storage, Cloud Functions, Authentication). Solo activos de `cliente-a` .
- **Metodología:** Reconocimiento → pruebas de autorización (anónimo y autenticado) → escalada de privilegios → remediación → reporte.

Resumen ejecutivo

Se identificaron **1 vulnerabilidad crítica**, **1 media** y **1 baja**, **todas remediadas durante el ejercicio**. Los controles de autorización centrales (RBAC en Firestore y Storage, App Check en Cloud Functions, protección de funciones administrativas) **resistieron todas las pruebas** de acceso no autorizado y de escalada de privilegios. Postura de seguridad general: **sólida**. Riesgo residual: **bajo**.

Hallazgos

ID	Severidad	Hallazgo	Estado
C-01	Crítica	Directorio `.git` público → código, historial y token de GitHub expuestos	Remediado
M-01	Media	Código de servidor expuesto (`functions/index.js` , `package.json`)	Remediado
B-01	Baja	Faltaban cabeceras de seguridad HTTP	Remediado

C-01 — Exposición de `.git` + credencial (Crítica)

- **Descripción:** la configuración de Firebase Hosting servía toda la carpeta del proyecto (`"public": "."`) y el filtro de exclusión no cubría los archivos dentro de `.git/` . En consecuencia, `https://.../.git/config` , `/.git/index` , `/.git/logs/HEAD` , refs y objetos eran accesibles públicamente, permitiendo reconstruir el repositorio privado. Además, `.git/config` contenía un token de GitHub (`http.extraheader` , tipo `ghs_`).
- **Impacto:** filtración de código fuente privado, historial completo y una credencial de acceso a GitHub.
- **Evidencia:** `GET /.git/config` → 200 con `AUTHORIZATION: basic <token>` .
- **Remediación:** se excluyeron `.git/**` , `functions/**` , `.github/**` , `package.json` del Hosting y se redeployó. Verificado: los recursos ahora devuelven 404.

- **Mitigantes:** el token era efímero (`ghs_`, corta duración) y la configuración local ya usaba Git Credential Manager (sin token embebido).

M-01 — Código de servidor expuesto (Media)

- `functions/index.js` y `package.json` eran servidos públicamente. **Remediado** (excluidos del Hosting; 404).

B-01 — Cabeceras de seguridad ausentes (Baja)

- Solo estaba HSTS. Se agregaron: `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` (geo/cámara/micrófono/pago deshabilitados) y `Content-Security-Policy: frame-ancestors 'self'`. **Remediado.**

Controles validados (efectivos)

- **RBAC en Firestore:** un atacante anónimo y un usuario con rol `editor` **no** pueden leer `consultas`, `procesos_insolvencia` ni `auditoria`.
- **Segregación por rol en escritura:** `editor` solo puede escribir en `posts` (blog); `guias` y `servicios` denegados.
- **Escalada de privilegios:** un `editor` autenticado **no puede** asignarse el rol admin (`adminAsignarRol` bloqueado) ni listar usuarios.
- **App Check:** las Cloud Functions rechazan llamadas sin atestación válida.
- **Storage:** los documentos de expedientes son inaccesibles (listar/descargar) sin rol autorizado.
- **MFA:** las cuentas exigen segundo factor (TOTP).

Recomendaciones (hardening futuro)

1. **CSP estricta** de scripts (requiere migrar scripts inline a nonces/hashes).
2. **Actualizar dependencias** vulnerables (`firebase-admin` / `firebase-functions`).
3. **Validación de datos** en reglas (tipos y tamaños en `consultas` / `procesos`).
4. **Servir desde carpeta dedicada** (`public/`) en lugar de la raíz (allowlist en vez de blocklist).
5. **Rotación periódica** de secretos y monitoreo de abuso/costos.

Conclusión

La arquitectura aplica correctamente los principios de **mínimo privilegio** y **defensa en profundidad**. La vulnerabilidad crítica fue un problema de **configuración de despliegue** (no de la lógica de la aplicación) y se corrigió durante el ejercicio. Se recomienda repetir el pentest tras cambios mayores y aplicar las recomendaciones de hardening.

Ejercicio realizado sobre activos propios y con autorización.